

PATRONES APRENDIDOS Y OPERACIÓN SIN CONEXIÓN EN ESPAÑOL: UNA INTERFAZ DE LENGUAJE NATURAL CENTRADA EN EL HUMANO CON SOPORTE MULTIDISTRIBUCIÓN PARA GNU/LINUX

Jose Ramon Aragon Toledo y Ricardo Ruiz Rodríguez

Universidad Tecnológica de la Mixteca (UTM)

Carretera a Acatlima Km. 2.5, Huajuapán de León, Oaxaca 69000, México

aatr010423@gs.utm.mx, rruiz@gs.utm.mx

RESUMEN

GNU/Linux sostiene aproximadamente el 90 % de la infraestructura en la nube, y sin embargo su interfaz de línea de comandos excluye a la mayoría de los usuarios no expertos, y esa exclusión golpea con más fuerza a las comunidades cuya primera lengua no es el inglés. Este artículo presenta una versión extendida de BOTAS, una interfaz conversacional que traduce español hablado a comandos de GNU/Linux válidos y ejecutables, bajo principios de Inteligencia Artificial Centrada en el Humano. Ampliando nuestro reporte previo de congreso, describimos tres incorporaciones—un nivel de patrones aprendidos que convierte cada petición resuelta por el modelo de lenguaje en una plantilla parametrizada reutilizable, recuperada mediante TF-IDF y similitud coseno, y rellena a través de anclas léxicas adaptativas; un modo completamente sin conexión en el que la transcripción, la interpretación y la síntesis se ejecutan de forma local; y soporte multidistribución verificado mediante instalación desatendida en tres gestores de paquetes—y reportamos cuatro evaluaciones. En un banco de pruebas de 250 tareas evaluado por un arnés automatizado bajo un criterio estricto de concordancia de acción, el sistema alcanza 78.0 % de cobertura y 87.6 % de precisión cuando actúa, sin falsos negativos de seguridad entre las operaciones generadas que modifican el estado; un segundo modelo con precisión casi idéntica se abstiene de actuar tres veces más seguido, lo que convierte la elección del modelo en una decisión de diseño de interacción y no de mera adquisición. Un estudio de usabilidad ($n = 14$) encontró que la voz es de 2.1 a 2.5 veces más lenta que el mouse y el teclado en tareas que el entorno gráfico ya resuelve bien. Una comparación controlada intra-sujeto ($n = 7$) contra la línea base que el sistema realmente desplaza—un usuario no experto averiguando qué escribir—la encontró 2.80 veces más rápida, una reducción del 66.6 %, con todos los participantes más rápidos (Wilcoxon exacta $p = .0156$, $d_z = 3.63$). Leídos en conjunto, estos resultados ubican la contribución: este tipo de interfaces no reemplaza la manipulación directa, sino que sustituye a la terminal donde la manipulación directa deja a los usuarios sin servicio, y su beneficio escala con la distancia respecto a la experticia ($r = 0.97$).

PALABRAS CLAVE

IA Centrada en el Humano, Interfaz de Lenguaje Natural, GNU/Linux, Interacción por Voz, Evaluación de Usabilidad, Equidad Digital

1. INTRODUCCIÓN

GNU/Linux sostiene aproximadamente el 90 % de la infraestructura en la nube, corre en el 96.3 % del millón de servidores más importantes del mundo, y domina el mercado de la supercomputación (W3Techs, 2023; Top500.org, 2023). A pesar de esta ubicuidad, la línea de comandos que desbloquea su potencial sigue siendo inaccesible para la mayoría. Quien quiera listar archivos mayores a 20 MB debe aprender que la herramienta se llama `find`, que el tamaño se escribe `-size +20M`, y que sin `-type f` también devuelve directorios. Una intención de diez segundos se vuelve una expedición por la documentación: los usuarios novatos dedican entre ocho y quince minutos a construir comandos básicos (Norman, 2013). Jenson (2024) argumenta que el estancamiento resultante “no es un límite técnico sino una capitulación de diseño”: las interfaces gráficas sacrifican la composición que hace valioso a GNU/Linux, las páginas de manual asumen experticia, y los guiones prefabricados no anticipan toda necesidad. La distancia entre saber qué se quiere y saber cómo decírselo a la computadora sigue siendo amplia; Shneiderman and Plaisant (2010) ya había propuesto el lenguaje natural como puente.

En la versión de congreso (Aragon Toledo and Ruiz Rodríguez, 2026) presentamos BOTAS (*Bot de Operaciones y Tareas Automatizadas del Sistema*), una interfaz conversacional que traduce español hablado o escrito a comandos de GNU/Linux válidos y ejecutables. Encarna los principios de Inteligencia Artificial Centrada en el Humano (HCAI, por sus siglas en inglés) (Shneiderman, 2020) a través de cinco compromisos: amplificar en lugar de reemplazar la capacidad humana; explicar qué hace cada comando y por qué; mantener al usuario en control mediante confirmación previa a la ejecución; priorizar la seguridad con detección determinista de operaciones peligrosas; y construir competencia en lugar de dependencia. Está dirigido a hispanohablantes—más de 500 millones de personas desatendidas por interfaces anglocéntricas—porque en América Latina las barreras lingüísticas y económicas se agravan mutuamente en las instituciones educativas (Warschauer, 2004; CEPAL, 2023). Ese artículo describía una arquitectura y una evaluación exploratoria. Desde entonces el sistema adquirió un nivel de aprendizaje, un modo sin conexión y soporte multidistribución verificado, y pasó por dos estudios con usuarios nuevos. Este artículo reporta todo eso: un *nivel de patrones aprendidos* que convierte cada petición resuelta por el modelo en una plantilla reutilizable, recuperada por TF-IDF y similitud coseno y rellena por anclas léxicas adaptativas, de modo que una paráfrasis ya resuelta nunca vuelve al modelo (§3.2); un *modo sin conexión*, con las restricciones que hacen viable un modelo de 6.7 mil millones de parámetros en una GPU de seis gigabytes (§§3.5, 4.3); *soporte multidistribución validado* mediante instalación desatendida en tres gestores (§4.4); un *estudio de usabilidad* (n = 14) que contrasta voz contra entorno gráfico bajo dos modelos, y una *comparación controlada* (n = 7) contra la línea base real, un usuario averiguando qué escribir (§§4.5, 4.6); y una *reevaluación* de la generación de comandos, reportada con transparencia contra las cifras publicadas (§4.1).

2. TRABAJOS RELACIONADOS

2.1 Interfaces de lenguaje natural para la línea de comandos

La línea de comandos persiste por su capacidad de composición, pero Nielsen (1994) mostró que exige 10–20 horas de entrenamiento frente a 2–4 para las interfaces gráficas, brecha que solo se ha ampliado con más comandos por distribución. NL2Bash (Lin et al., 2018) fue pionero en el mapeo de lenguaje natural a comandos de shell con 36 % de coincidencia exacta, pero no abordó ni la ejecución ni la seguridad. Desde entonces los modelos grandes de lenguaje han mejorado notablemente la generación de comandos (Brown et al., 2020), pero los asistentes generales como ChatGPT (OpenAI, 2023) siguen siendo de solo sugerencia: el usuario copia, pega y espera, sin validación previa a la ejecución. Herramientas orientadas a terminal como ShellGPT (Shchekotov, 2024) añaden ejecución, pero típicamente sin capa de seguridad, sin retroalimentación educativa y sin memoria alguna de lo que el usuario ha pedido antes. Reutilizar respuestas previas para peticiones semánticamente equivalentes es una estrategia establecida de reducción de costos—GPTCache (Bang, 2023) indexa respuestas cacheadas por similitud de *embeddings*—pero una caché semántica devuelve una *respuesta* previa, mientras que el nivel de la Sección 3.2 almacena una *plantilla de comando parametrizada* con el rol semántico de cada variable, de modo que una paráfrasis que nombra una carpeta distinta se sigue resolviendo localmente, con la nueva carpeta sustituida. Su motor de recuperación es deliberadamente clásico (Manning et al., 2008), porque debe ejecutarse en Perl puro en menos de cinco milisegundos, sin modelo y sin red.

2.2 Asistentes de voz para GNU/Linux

Los asistentes de voz de software libre para escritorio tienen una historia accidentada. Jasper (Saha et al., 2014) estableció el patrón de escucha continua y prioridad fuera de línea, pero es anterior a los modelos de lenguaje modernos y dependía de módulos de intención escritos a mano; Mycroft, el intento más visible, cesó operaciones en 2023 tras una disputa de patentes (Dobberstein, 2023), y su continuación comunitaria OpenVoiceOS (OpenVoiceOS, 2024) preserva la arquitectura basada en *skills*. Todos comparten una misma restricción estructural: la capacidad está acotada por las habilidades que alguien escribió de antemano. BOTAS invierte esa relación—el modelo de lenguaje aporta la interpretación abierta, mientras que una capa determinista, no el modelo, aporta las garantías.

2.3 IA centrada en el humano y equidad digital

Shneiderman (2020; 2022) establecieron HCAI como un marco que enfatiza transparencia, seguridad y empoderamiento del usuario, argumentando que la alta automatización y el alto control humano son complementarios y no opuestos; Amershi et al. (2019) derivaron de ahí lineamientos de diseño que informan varias de las decisiones descritas más adelante. Luger and Sellen (2016) encontró que la ausencia de retroalimentación de estado es una de las fuentes más confiables de frustración en agentes conversacionales, lo que motivó el indicador de cuatro estados de la Sección 3.6. En cuanto a la equidad, Warschauer (2004) documenta cómo las

barreras técnicas afianzan las desigualdades, y Wobbrock et al. (2011) propuso el diseño basado en habilidades, en el que las interfaces se adaptan a los usuarios y no al revés—una filosofía que BOTAS aplica al aceptar el español hablado corriente como modalidad primaria.

3. ARQUITECTURA DEL SISTEMA

BOTAS está implementado en Perl—por su integración con Unix, su fuerza en procesamiento de texto y sus *bindings* maduros de GTK3—en doce módulos que suman unas 16 500 líneas, más un proceso embebido de Python para los componentes acústicos. La Figura 1 muestra el flujo: la entrada llega por la interfaz gráfica, el botón de micrófono o el detector de palabra de activación en segundo plano; se interpreta mediante un flujo de tres niveles que escala solo cuando los niveles más baratos fallan; el comando resultante pasa por una capa de seguridad determinista; y la ejecución se reporta de vuelta en lenguaje sencillo y, opcionalmente, por voz.

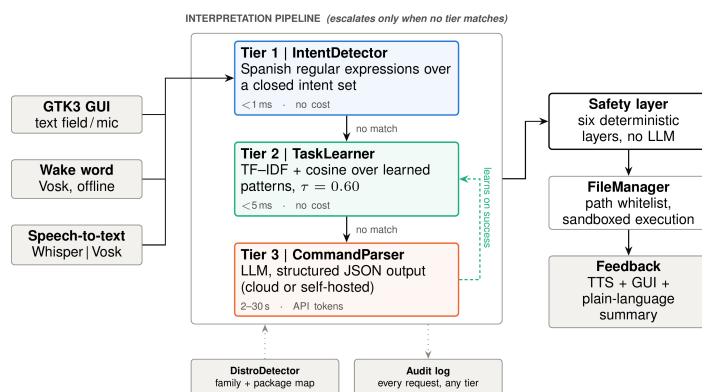


Figura 1. Arquitectura del sistema. El flujo de interpretación escala de expresiones regulares a patrones aprendidos y de ahí al modelo de lenguaje, de modo que solo se incurre en costo cuando los niveles más baratos no pueden responder. La ruta de retorno punteada es el ciclo de aprendizaje: cada petición que el modelo resuelve con éxito se convierte en un patrón que el segundo nivel puede atender la próxima vez

3.1 El flujo de interpretación de tres niveles

La decisión central de diseño es que la interpretación no es un acto único. El orquestador primero deduplica la transcripción—Whisper a veces repite un enunciado completo dos o tres veces en grabaciones largas, inflando el costo y corrompiendo el emparejamiento de patrones—y luego intenta tres mecanismos en orden ascendente de costo. Cada nivel devuelve la misma estructura JSON, de modo que la validación de seguridad, la ejecución y el registro son indiferentes a cuál nivel produjo el comando. Llamamos a esto *niveles* y no *capas* porque no se recorren en secuencia: una petición se detiene en el primer nivel que la resuelve. *Capa* se reserva para el mecanismo de seguridad de la Sección 3.3, por el cual sí pasa todo comando generado.

El **Nivel 1** atiende peticiones sin comprensión lingüística: la hora, la fecha, la memoria libre,

abrir una aplicación. `IntentDetector.pm` compara contra expresiones regulares curadas en español y, ante un acierto, responde en menos de un milisegundo sin costo. Su cobertura es deliberadamente estrecha—intenciones conversacionales y de información inmediata, no generación de comandos—y la Sección 4.2 cuantifica qué tan estrecha.

El **Nivel 3**, descrito antes que el Nivel 2 porque este último aprende de aquel, envía la petición a un modelo de lenguaje mediante un *prompt* de sistema (Wei et al., 2022) que devuelve JSON con cuatro campos: `action` de un conjunto cerrado, un diccionario de `parameters`, un `tts_response` en lenguaje sencillo para el resumen educativo y la síntesis de voz, y una bandera `requires_confirmation`. Una acción, `clarify`, lleva una pregunta de seguimiento en vez de un comando: así se abstiene el sistema de adivinar. El *prompt* también lleva el directorio de trabajo, la familia de distribución y los últimos diez intercambios, permitiendo resolver anáforas como “*esos*”. Hay tres proveedores: la API de OpenAI, la de DeepSeek—compatible con la de OpenAI, así que integrar es cambiar la URL base—y cualquier servidor autoalojado con el mismo protocolo, que es como opera el modo sin conexión. La salida estructurada elimina la ambigüedad del texto libre y hace determinista todo lo que sigue.

3.2 Nivel 2: patrones de tarea aprendidos

`TaskLearner.pm` (1 295 líneas, 35 subrutinas) es el componente añadido desde la versión de congreso: la memoria de largo plazo del sistema. Su premisa es que las peticiones que un usuario repite son precisamente las costosas, y que una tarea ya resuelta por el modelo no debería enviársele de nuevo. La Figura 2 muestra sus dos rutas.

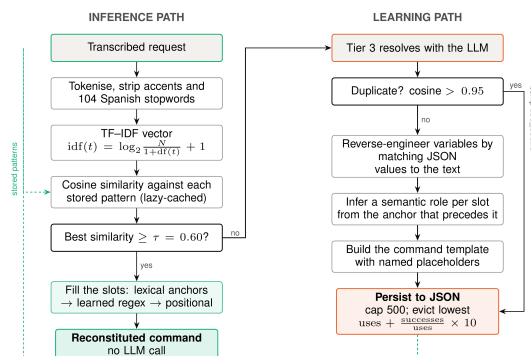


Figura 2. Las dos rutas del nivel de patrones aprendidos. A la izquierda, una petición entrante se compara contra los patrones almacenados y, por encima del umbral de confianza, se reconstituye en un comando sin llamada alguna al modelo. A la derecha, una petición que el modelo acaba de resolver se convierte en una plantilla parametrizada y se persiste

3.2.1 Recuperación

El motor de similitud es Perl puro sin dependencia externa alguna. La entrada se convierte a minúsculas, se le eliminan acentos y puntuación, y se filtra contra 104 palabras vacías del español.

Los términos se ponderan con una frecuencia inversa de documento suavizada,

$$\text{idf}(t) = \log_2 \left(\frac{N}{1 + \text{df}(t)} \right) + 1, \quad (1)$$

donde N es el número de patrones almacenados y $\text{df}(t)$ el número que contiene el término t ; el vector disperso resultante se compara contra cada patrón por similitud coseno. Los vectores se calculan de forma perezosa y se cachean, invalidándose ante cualquier cambio del conjunto. La mejor coincidencia se acepta en un umbral configurable, $\tau = 0,60$ por defecto, ajustado empíricamente: si es menor, las coincidencias cercanas producen falsos positivos, con riesgo de ejecutar el comando equivocado; si es mayor, solo coinciden repeticiones casi textuales, desperdiciando el aprendizaje.

3.2.2 Adquisición y relleno de plantillas

Cuando el Nivel 3 resuelve una petición, el orquestador entrega el par (petición, comando) a `learn()`. Una verificación de duplicados descarta todo lo que supere coseno 0.95, incrementando en su lugar un contador de uso. En caso contrario, el módulo realiza lo que llamamos ingeniería inversa de variables: busca en la petición los valores concretos que el modelo colocó en el JSON—un nombre de archivo, un destino, una cadena de búsqueda—y sustituye cada uno por un marcador de posición con nombre, produciendo una plantilla en lugar de un comando fijo.

La dificultad es que la siguiente invocación no estará redactada igual. Un patrón aprendido de “*Crea una carpeta llamada Matemáticas en Documentos*” debe seguir sirviendo a “*Quiero que crees una carpeta llamada Backup dentro de la carpeta Documentos*”, más larga, con preposición distinta y orden distinto. Una expresión regular rígida falla aquí, y una heurística posicional captura fragmentos espurios como “*la carpeta Documentos*”. La solución es vincular cada variable a un *rol semántico* y reconocer ese rol por las expresiones en español que lo introducen, sin importar la posición. El módulo define seis roles sobre 62 anclas: *location* (*dentro de la carpeta, en*), *name* (*llamada, el archivo*), *new_name* (*renómbrala a*), *source* (*mueve, copia*), *query* (*busca sobre*) y *pattern* (*que contengan*). La extracción normaliza preservando acentos y mayúsculas, busca exhaustivamente cada ancla de cada rol, resuelve solapamientos por posición ascendente y longitud descendente—así *dentro de la carpeta* gana sobre el *dentro de* embebido—, extrae cada valor como el fragmento entre un ancla y la siguiente, limpia palabras vacías y ruido de dominio solo en los *extremos* de un valor, y asigna valores a variables por rol. En la paráfrasis anterior, se detectan las anclas *llamada* y *dentro de la carpeta*, se descarta el *dentro de* embebido, y las variables reciben *Backup* y *Documentos*—produciendo `~/Documentos/Backup` sin llamada alguna al modelo.

La extracción por anclas es la primera de tres estrategias, de la más flexible a la más rígida: si falla, recurre a la expresión regular aprendida junto con el patrón, y luego a una heurística posicional. Si las tres fallan, la petición escala al modelo, que enseña al módulo la nueva variante. Los patrones tienen un tope de 500; al desbordarse se descarta el de menor índice de utilidad, usos + (xitos/usos) × 10, favoreciendo los patrones que son a la vez frecuentes y confiables.

3.3 Seguridad

Dado que el sistema ejecuta operaciones de archivos derivadas de la salida del modelo, ninguna garantía de seguridad puede depender del modelo. Seis capas deterministas se interponen entre

una petición interpretada y sus efectos. Un bloque `BEGIN` elimina variables del cargador que podrían forzar bibliotecas no autorizadas. Cada ruta resuelta se verifica contra una lista blanca de raíces permitidas: un recorrido como `../../etc/passwd` lanza excepción en vez de alcanzar el sistema de archivos. Cada operación se clasifica como segura, de advertencia o peligrosa mediante reglas basadas en el tipo de acción y sus parámetros—expresiones regulares y heurísticas, nunca inferencia del modelo, de modo que el comportamiento es consistente y auditable. Las operaciones de advertencia y peligrosas requieren confirmación explícita. Una sola instrucción puede afectar como máximo cien archivos. Y cada interacción se registra con su marca de tiempo, texto original, intención detectada, resultado y nivel de origen. Esto es defensa en profundidad y no una sola verificación, y su alcance debe declararse con claridad: detecta categorías *conocidas* de operación peligrosa. No puede anticipar un vector de ataque novedoso, y no razona sobre el contexto—eliminar un directorio se clasifica igual sin importar si contiene archivos de caché o una tesis.

3.4 Detección de distribución y soporte multidistribución

La fragmentación entre familias de distribuciones es fuente conocida de confusión: `apt install foo` falla en Fedora, que quiere `dnf`, y en Arch, que quiere `pacman`. `DistroDetector.pm` analiza `/etc/os-release`, leyendo `ID`, `ID_LIKE` y `VERSION_ID`, y clasifica el equipo en una de seis familias—`debian` (`apt-get`), `fedora` (`dnf`), `arch` (`pacman`), `suse` (`zypper`), `alpine` (`apk`) y `void` (`xbps-install`)—recurriendo a sondear gestores de paquetes cuando ese archivo no está disponible. Un diccionario mapea 12 nombres genéricos de dependencias, escritos en convención Debian, a los equivalentes de cada familia, con `cpanm` como respaldo común cuando no existe paquete nativo. La familia detectada se cachea y se inyecta tanto en el *prompt* de sistema del modelo como en los manejadores del primer nivel, de modo que todo comando generado es adecuado para la máquina en la que se ejecutará.

La misma tabla impulsa la instalación: un instalador universal clasifica el equipo, traduce las dependencias, instala mediante el gestor nativo con `cpanm` como respaldo, escribe configuración inicial y registra una entrada de escritorio; un script complementario maneja el modelo de palabra de activación sin conexión. Un directorio portable construye un `.deb` independiente de la arquitectura—todo es Perl y Python interpretados, así que un solo paquete sirve tanto para `amd64` como para `arm64`—y la misma maquinaria produce un `.rpm`.

3.5 Operación en línea y sin conexión

BOTAS opera en dos modos que intercambian tres componentes entre nube y local, conmutables en tiempo de ejecución. La transcripción pasa de OpenAI Whisper a Vosk local (Alpha Cephei, 2023; Radford et al., 2023); la interpretación de `gpt-4o` o `deepseek-chat` a `deepseek-coder-6.7b` local por LM Studio; la síntesis de voz del servicio de OpenAI a eSpeak. El intercambio es latencia y costo contra privacidad: 2–5 segundos y tokens de API en línea, 5–45 segundos y ningún costo sin conexión, donde ningún audio ni texto sale de la máquina. Un único *callback* propaga el cambio a los tres módulos sin necesidad de reiniciar. Un componente nunca cambia: el detector de palabra de activación corre sobre Vosk en ambos modos, de modo que la escucha pasiva nunca genera tráfico de red—el sistema no está “siempre escuchando” hacia los servidores de nadie más.

Los modelos locales pequeños no son versiones más lentas de los remotos grandes, y la ruta sin conexión se adapta a ellos: el historial se trunca a los últimos dos intercambios en lugar de diez; el límite de tokens se reduce, porque un modelo pequeño sin restricción divagará hasta el límite; el tiempo de espera HTTP sube a 300 segundos; un *prompt* de sistema más corto reemplaza al completo; y la imposición del modo JSON, que ofrecen los proveedores en la nube, se reemplaza por una limpieza heurística que extrae el fragmento entre la primera llave y la última.

3.6 Flujo de voz y retroalimentación de estado

La operación manos libres funciona así: el detector de palabra de activación escucha sin conexión con una latencia menor a 200 ms; al activarse se abre una ventana de atención de 30 segundos; el enunciado se transcribe y se procesa exactamente igual que la entrada escrita; y la respuesta se pronuncia por voz. Atendiendo al hallazgo de Luger and Sellen (2016) de que la ausencia de retroalimentación de estado frustra a los usuarios, la interfaz lleva un indicador de cuatro estados—en espera, escuchando, grabando, procesando—distinguido por color y posición, y cada comando ejecutado va acompañado de una declaración en lenguaje sencillo de la intención y, para operaciones marcadas, del riesgo. La Figura 3 muestra ambas superficies: la ventana principal, donde el usuario ve qué está haciendo el sistema, y el diálogo de confirmación de la Sección 3.3, donde nada procede sin una decisión explícita.



Figura 3. Las dos superficies mediante las cuales el usuario conserva el control, en su español nativo, con los espacios intermedios elididos. A la izquierda, la ventana principal: (A) registro de actividad, (B) hablar al presionar, (C) modo de atención manos libres, (D) ayuda, (E) tutorial. A la derecha, el diálogo que aparece antes de cualquier operación de advertencia o peligrosa: (A) qué se hará y dónde, (B) el marcador de advertencia, (C) cancelar o ejecutar

4. EVALUACIÓN EXPERIMENTAL

El sistema se evaluó en cuatro líneas: un banco de pruebas de generación de comandos y detección de riesgo; una prueba de viabilidad del modo sin conexión; un estudio de usabilidad; y una comparación controlada de tiempo contra la línea base que el sistema desplaza. Los estudios con personas usaron consentimiento informado, identificadores codificados y derecho a retirarse.

4.1 Una nota sobre las cifras publicadas en la versión de congreso

La versión de congreso reportó 89 % de precisión sobre 200 tareas, aceleración de $9.4\times$ frente a lo manual, y 30 % de peticiones resueltas localmente—cifras de verificación manual del desarrollador, cronometraje informal, y una estimación de qué tan seguido lo rutinario cae en el primer nivel. Las tres desde entonces han sido reemplazadas por mediciones tomadas con mejores instrumentos, y las cifras cambiaron; reportamos el cambio de manera explícita en lugar de sustituir un conjunto por otro en silencio.

Cambiaron tres cosas: el conjunto creció de 200 a 250 tareas, evaluado por un arnés automatizado que ejecuta el flujo real; el criterio se endureció a *concordancia de acción* sobre un subconjunto *dentro de alcance*, correcta solo si la acción es funcionalmente equivalente a la esperada, y las tareas fuera del repertorio dejan el denominador en vez de calificarse con generosidad. Y el cronometraje informal fue reemplazado por la comparación de la Sección 4.6. Bajo el criterio más estricto la precisión general es menor, la cifra de cobertura local mucho menor, y la aceleración menor pero, por primera vez, respaldada estadísticamente. Las cifras a continuación reemplazan a las de Aragon Toledo and Ruiz Rodríguez (2026).

4.2 Generación de comandos sobre el conjunto de 250 tareas

El conjunto de evaluación contiene 250 peticiones en español en ocho categorías, cada una con un comando y un nivel de riesgo esperados (219 seguras, 23 de advertencia, 8 peligrosas); las corridas se hicieron en Ubuntu 22.04 LTS. Con `deepseek-chat` el arnés analizó las 250 respuestas sin error y coincidió en la acción en el 78.0 % de las 127 tareas dentro de alcance. La concordancia fue completa en control de procesos, búsqueda web y apertura de aplicaciones, alta en información del sistema (94.1 %) y en redes (92.3 %), y más baja en gestión de archivos (63.2 %) y creación de proyectos (45.0 %). Los errores residuales son en su mayoría benignos—leer *copia* como un movimiento, o “*muestra el contenido*” como una edición—y la categoría más débil refleja al modelo alternando entre dos acciones adyacentes en lugar de malinterpretar la petición.

Ejecutar las mismas 250 tareas con `gpt-4o` dio un resultado más interesante que una clasificación (Tabla 1); donde difirieron, la diferencia fue sistemática: ante una petición que omite una ruta—“*renombra archivo.txt a documento.txt*”—`gpt-4o` devuelve `clarify` donde `deepseek-chat` asume un valor por defecto, bajando su concordancia a 56.7 %. Pero al sacar del denominador los repliegues a `clarify` de cada modelo, los dos son indistinguibles y el orden se invierte: ambos superan 87 % de precisión al actuar. La brecha no está en la competencia sino en el temperamento. Las divergencias más amplias fueron en gestión de archivos (18.4 % frente a 63.2 %) y permisos (0 % frente a 75.0 %), ambas categorías donde debe inferirse un destino; en búsqueda web y apertura de aplicaciones ambos alcanzaron 100 %.

De las 250 peticiones, 28 produjeron una acción que modifica el estado. Cinco se materializaron como operaciones destructivas, todas eliminaciones, y las cinco activaron confirmación: sin falsos negativos de seguridad, para ninguno de los dos modelos en la nube. Las peticiones peligrosas en un *shell* pero fuera del repertorio (`reboot`, `pkill`, `systemctl stop`) se enrutaron a una acción informativa o a `clarify` en lugar de ejecutarse.

Medido sobre las mismas tareas, el detector local respondió 11, o 4.4 %—muy por debajo del 30 % estimado en la versión de congreso. La discrepancia es instructiva: el primer nivel nunca fue

Tabla 1. Cobertura frente a precisión para los dos modelos en la nube sobre el subconjunto dentro de alcance (127 de 250 tareas). La precisión al actuar es aciertos entre tareas dentro de alcance menos repliegues a clarificación. Los dos modelos concuerdan en la acción en el 64.8 % de las 250 tareas

Métrica (dentro de alcance)	deepseek-chat	gpt-4o
Concordancia de acción (cobertura)	78.0 %	56.7 %
Repliegue a <code>clarify</code>	11.0 %	36.2 %
Precisión al actuar	87.6 %	88.9 %

diseñado para generar comandos, sino para resolver intenciones conversacionales frecuentes en el uso diario pero casi ausentes de un banco de tareas de generación. Sus aciertos se concentran donde uno lo predeciría—40 % en apertura de aplicaciones, 11.1 % en información del sistema, 5.4 % en redes, nada en el resto. El mecanismo que genuinamente reduce las llamadas al modelo para trabajo real es el segundo nivel, a cuya contribución volvemos en la Sección 5.4.

4.3 Viabilidad sin conexión

Para acotar lo que el sistema puede hacer sin red, se corrió un subconjunto de 100 tareas—las 50 más recientes más una muestra estratificada de 50 del resto, ponderada hacia riesgo—contra un modelo local, conciliando tres restricciones. Un modelo de 1.3 mil millones de parámetros se descartó por producir JSON malformado y divagar en prosa; siete mil millones resultó el piso para salida estructurada válida. Los modelos de razonamiento se descartaron porque sus trazas de deliberación multiplican la latencia más allá de lo tolerable en voz. Y ajustarse a seis gigabytes de video significó cuantización Q4_K_M en vez de Q5 y contexto fijado en 4 096 tokens, pues el modelo carece de atención de consultas agrupadas y su caché clave-valor cuesta medio megabyte por token. Descargar las 32 capas a la GPU elevó la generación de 1.7 a 21 tokens por segundo.

La configuración resultante—`deepseek-coder-6.7b-instruct`, Q4_K_M, contexto de 4 096 tokens, en una GTX 1060 de 6 GB—produjo JSON válido en las 100 tareas y coincidió en la acción en 67.5 % de las 77 dentro de alcance, frente a 77.2 % de `deepseek-chat` y 64.6 % de `gpt-4o`. Dos advertencias importan más que ese titular. La latencia subió de aproximadamente 2–5 a aproximadamente 5–45 segundos; y el modelo local nunca pidió aclaración, así que ante una petición peligrosa fuera de su repertorio la reinterpreto como alguna acción que sí conocía, produciendo dos falsos negativos de seguridad donde ambos modelos en la nube no produjeron ninguno. La capa determinista los absorbió—precisamente el caso para el que existe—pero el hallazgo matiza el modo sin conexión en lugar de respaldarlo sin reservas. Estas cifras establecen que el enfoque ya funciona en hardware modesto, no cuál es su techo.

4.4 Validación multidistribución

Soportar seis familias es una propiedad del código; funcionar en ellas es una afirmación que necesita ponerse a prueba. El instalador se ejecutó en máquinas virtuales limpias, en cada caso con un solo comando—clonar el repositorio, ejecutar el instalador—sin intervención manual posterior. Se verificaron cinco criterios: detección correcta de la familia, gestor y nombres de paquete correctos, resolución de dependencias de Perl (con respaldo a `cpanm`), una aplicación funcional,

y—deliberadamente más exigente—la instalación de la palabra de activación sin conexión, Vosk más su modelo acústico en español. Se probó en cuatro distribuciones limpias: Ubuntu 22.04 LTS (debian), Kali *rolling* (debian), Fedora 44 Workstation (fedora) y EndeavourOS/Manjaro (arch); el instalador, la aplicación y la palabra de activación funcionaron en las cuatro, salvo un resultado parcial en Kali donde la red de prueba bloqueó la descarga del modelo acústico, no un fallo del instalador.

El instalador se adaptó de forma autónoma en los cuatro casos. Fedora es el resultado más revelador: una máquina de otra familia, con gestor y nombres de paquete distintos, pasó de instalación desnuda a aplicación funcional—palabra de activación incluida—sin un solo paso manual. El ejercicio también sacó a la luz cuatro problemas de robustez invisibles dentro de la familia de desarrollo, todos ya corregidos: caché de `dnf` fría abortando el lote, `HOME` apuntando a `/root` bajo `sudo`, un paquete fallido abortando todo el lote, y el soporte HTTPS de Perl ausente en varios conjuntos base. Cuatro de seis familias, cubriendo los tres gestores más extendidos, están verificadas; las otras dos descansan en el mismo diccionario pero no se han ejercitado.

4.5 Estudio de usabilidad

Se realizaron quince sesiones con la comunidad universitaria; la primera fue piloto y se excluye, dejando catorce participantes (ocho hombres, seis mujeres; edad media 25.9, rango 22–32; once usuarios de Windows, dos de macOS, uno de GNU/Linux). El diseño fue intra-sujeto para la modalidad—cada tarea primero con mouse y teclado, luego por voz—y entre sujetos para el modelo de lenguaje, asignándose los participantes alternadamente por orden de llegada a `deepseek-chat` (siete) o `gpt-4o` (siete). Ni a los participantes ni a los observadores se les dijo qué modelo estaba en uso.

Se presentaron tres tareas como escenarios en tarjetas, sin prescribir las palabras: crear una carpeta de curso con seis subcarpetas de materia (T1); localizar un PDF en descargas y moverlo a una de ellas (T2); y hacer una búsqueda web por voz (T3). Las sesiones siguieron un protocolo de pensar en voz alta, interviniendo el facilitador solo ante solicitud explícita o tras tres minutos sin progreso. Los datos provinieron de hojas de observación, cuestionarios posteriores a cada tarea y finales, y registros del sistema, analizados mediante codificación temática inductiva (Braun and Clarke, 2006), enmarcando efectividad, eficiencia y satisfacción como en International Organization for Standardization (2018).

4.5.1 Efectividad y eficiencia

El éxito se calificó con crédito parcial (1.0 completa, 0.5 parcial, 0 fallida). En el entorno gráfico todos los participantes completaron todas las tareas; por voz la tasa ponderada promedió 83.9 %, más alta en búsqueda web (92.3 %) y más baja al mover un archivo (78.6 %), y la voz fue más lenta en las tres tareas por factores entre 2.1 y 2.5 (Figura 4).

La dispersión importa más que las medias. En T1 la desviación estándar de la voz es de 202.4 segundos frente a una media de 248.1, con tiempos individuales de 61 a 773 segundos; las cifras correspondientes del mouse son 39.5 y 119.4. La observación explica por qué: las sesiones lentas no fueron lentas porque la tarea fuera difícil, sino porque los participantes estaban estableciendo confianza en una modalidad desconocida—pausando antes de hablar, releendo el resultado, reformulando una vez para ver qué pasaba. Estos son costos de primer contacto, no desempeño en régimen estable, lo que hace que las medias reportadas sean una cota superior conservadora.

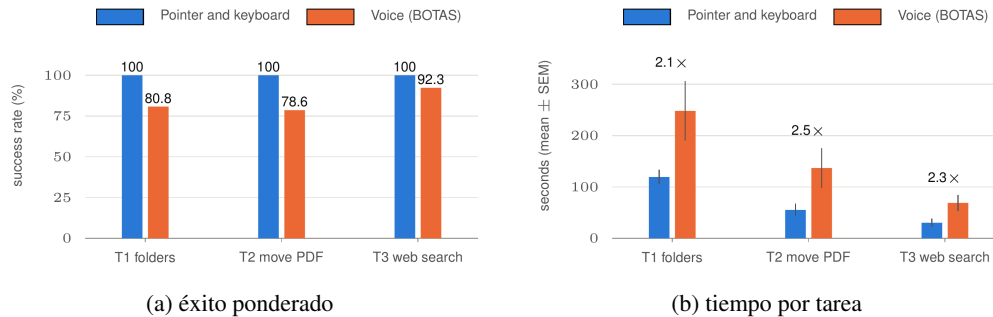


Figura 4. Estudio de usabilidad, $n = 14$. La voz queda por detrás del entorno gráfico en ambas medidas; las barras de error amplias reflejan qué tan distinto se comporta cada participante en el primer contacto con la modalidad

4.5.2 Satisfacción

A pesar de ser más lenta, la voz se calificó igual o ligeramente mejor que el entorno gráfico en facilidad percibida en las tres tareas (4.23 frente a 4.14 en T1; 4.38 frente a 3.67 en T2; 4.31 en ambos para T3, en una escala de cinco puntos). La Tabla 2 presenta el cuestionario final, con una calificación general de 8.21 sobre 10. El patrón entre dimensiones vale la pena leerse con atención: la facilidad de aprendizaje puntúa más alto con 4.54, comparable a hallazgos longitudinales sobre asistentes de voz entre usuarios no expertos (Jakob et al., 2025), mientras que la naturalidad del diálogo puntúa más bajo con 3.46. Los participantes encontraron el sistema fácil de aprender e incómodo para conversar—la misma asimetría que Berdasco et al. (2019) reportan para asistentes comerciales, lo que sugiere una propiedad de la interacción conversacional actual y no de esta implementación en particular.

Tabla 2. Cuestionario final de satisfacción. La calificación general está en escala de diez puntos, las demás dimensiones en escalas de cinco puntos. La condición A es `deepseek-chat`, la condición B es `gpt-4o`

Dimensión	General	DE	Condición A	Condición B	n
Calificación general (1–10)	8.21	0.97	7.86	8.57	14
Facilidad de aprendizaje	4.54	0.66	4.33	4.71	13
Accesibilidad percibida	4.23	1.01	4.33	4.14	13
Satisfacción general	3.62	0.77	3.67	3.57	13
Velocidad percibida	3.62	1.19	3.50	3.71	13
Naturalidad del diálogo	3.46	1.13	3.50	3.43	13

La frustración nunca alcanzó los niveles moderado o alto de la escala de observación; todo episodio registrado fue ausente o leve, concentrado en T2. La mayoría de los participantes prefirió la voz para crear carpetas (8 de 12 que expresaron preferencia) y para búsqueda web (7 de 13), mientras que la opinión se dividió de forma pareja al mover un archivo—la tarea con estructura más espacial y más amigable con el mouse. Con siete participantes por condición no hay afirmación inferencial disponible, pero la dirección fue consistente: `gpt-4o` lideró en ocho de doce dimensiones comparadas, incluyendo la efectividad por voz en las tres tareas.

4.6 La voz frente a buscar el comando

El estudio anterior compara la voz con el entorno gráfico, y en esa comparación la voz pierde en tiempo. Pero el entorno gráfico no es lo que BOTAS reemplaza. Un usuario que quiere los archivos de una carpeta por encima de cierto tamaño no lo encontrará en un explorador de archivos sin esfuerzo; buscará en la web, encontrará `find`, y lo adaptará. La pregunta relevante entonces no es si hablar vence a hacer clic, sino si hablar vence a *averiguar qué escribir*—una comparación que nunca se había instrumentado.

4.6.1 Diseño

Siete participantes (S1–S7), ninguno con experiencia previa en el estudio anterior o el sistema, realizaron una tarea bajo dos condiciones contrabalanceadas, con tope de cinco minutos. La tarea se leyó textualmente: “En la carpeta de Descargas hay muchos archivos. Muéstrame todos los archivos mayores a 20 MB.” La solución de referencia es `find ~/Descargas -type f -size +20M`, con la carpeta sembrada para que la respuesta correcta fuera un subconjunto propio. En la condición manual los participantes tenían un navegador y una terminal, y podían consultar lo que quisieran—motores de búsqueda, foros, tutoriales y asistentes conversacionales de propósito general—pero no BOTAS ni al observador: una línea base exigente, y en 2026 la única honesta. El cronometraje corrió desde el fin de la lectura del escenario hasta la aparición del listado correcto.

La muestra se filtró por inexperiencia: cinco nunca habían usado GNU/Linux, cinco nunca habían abierto una terminal, seis nunca habían oído de `find`; edades de 21 a 52 (media 28.7), ocupaciones de desarrollo de software a contabilidad y diseño gráfico. Siete pares es poco, y deliberado: la prueba exacta de rangos con signo de Wilcoxon (Wilcoxon, 1945) tiene un piso aritmético de $2/2^n$ en un valor p de dos colas, de modo que cinco pares no pueden bajar de 0.0625 y seis de 0.0312, mientras que siete alcanza 0.0156. Un análisis de sensibilidad ubica el efecto mínimo detectable en $d_z = 1.27$ para una potencia del 80 % con $\alpha = 0.05$ (Lakens, 2013).

4.6.2 Resultados

Todos fueron más rápidos con BOTAS, sin excepción ni empate (Figura 5); nadie alcanzó el tope de cinco minutos. La búsqueda manual promedió 141.5 segundos frente a 50.6 (Tabla 3): 90.9 s por tarea, reducción del 66.6 %, factor de 2.80 en medias y 2.95 en medianas. El caso menos favorable aun así ahorró 54.7 % del tiempo.

Tabla 3. Estadísticos descriptivos para las siete observaciones pareadas. Segundos salvo que se indique lo contrario

Condición	Media	DE	Mediana	Mín	Máx
Búsqueda manual de comandos	141.5	54.7	134.6	68.5	248.7
BOTAS por voz	50.6	30.8	45.6	14.2	112.6
Diferencia	90.9	25.1	91.0	54.3	136.1
Reducción (%)	66.6	8.3	64.1	54.7	79.3

La prueba exacta de rangos con signo de Wilcoxon da $W^+ = 28$, $W^- = 0$, $p = .0156$; la prueba de signos, 7 de 7, concuerda; la t pareada da $t(6) = 9.60$, $p < .001$; el tamaño del efecto es $d_z = 3.63$ y el intervalo de confianza del 95 % de la diferencia media es [67.8, 114.1] segundos.

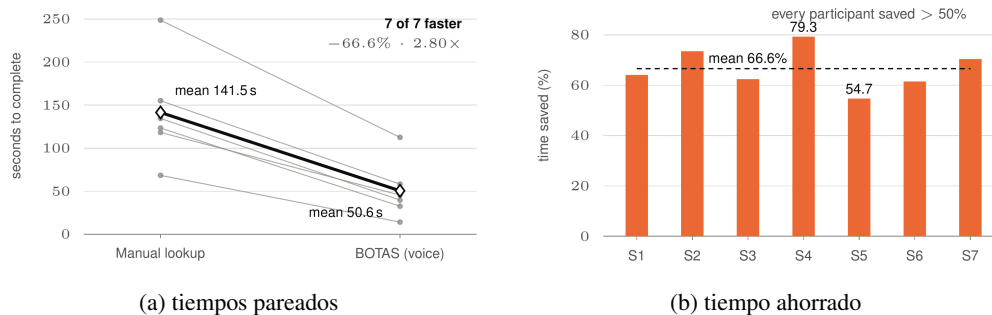


Figura 5. Búsqueda manual de comandos frente a BOTAS, $n = 7$, intra-sujeto y contrabalanceado. Las líneas grises son participantes individuales, la línea gruesa es la media. El efecto está presente en todos los participantes, incluyendo quien ya conocía el comando

$W^+ = 28$ es la suma completa de los rangos 1 a 7, de modo que $p = ,0156$ es el valor más pequeño que siete pares pueden producir: el diseño rindió la máxima evidencia disponible para él. El efecto observado está cerca de tres veces el efecto mínimo detectable, el intervalo no está ni cerca de cero, y la dirección es unánime. Lo que queda establecido es la existencia y dirección del efecto; la estimación puntual de $2.80\times$ conlleva incertidumbre real con este tamaño de muestra.

El contrabalanceo se comportó como debía: quienes usaron BOTAS primero ahorraron 71.4 % y quienes lo usaron después 62.9 %. Si la ventaja fuera artefacto de la repetición, el primer grupo debería mostrar la ventaja *menor*; ocurrió lo contrario. Las medidas de esfuerzo son tan reveladoras como los tiempos. Los siete completaron la tarea sin ayuda con BOTAS, frente a seis de siete manualmente—la excepción fue el participante de mayor edad, quien no sabía cómo abrir una terminal. Los participantes promediaron 2.43 intentos de comando manualmente frente a 1.00 por voz ($t(6) = 3.87$, $p = ,008$), e hicieron once consultas a motores de búsqueda, tutoriales o asistentes conversacionales frente a ninguna. Los siete llegaron al mismo comando en ambas condiciones, y BOTAS lo generó correctamente en todas las sesiones sin pedir aclaración ni una sola vez.

El obstáculo en la condición manual no fue escribir el comando sino descubrirlo. S1 obtuvo el comando correcto de un asistente conversacional y aun así tardó 141.9 segundos frente a 50.9: el costo está en todo el trayecto—abrir el navegador, formular la pregunta, leer la respuesta, cambiar de ventana, transcribir, ejecutar—no en la última tecla. Los tiempos de voz también son conservadores: dos participantes pausaron la captura para pensar cómo formular la petición, y esa vacilación está dentro de la medición.

El beneficio además se distribuye de forma desigual: la correlación entre el tiempo manual de un participante y lo que ahorró es $r = 0,97$ —entre más lejos está de conocer el comando, más vale el sistema para esa persona. La contadora sin experiencia en terminal tardó más manualmente (248.7 segundos, cinco intentos, cuatro búsquedas web, y la única solicitud de ayuda) y ahorró más en términos absolutos, 136.1 segundos. Aun así S4—un desarrollador junior que ya conocía `find` y lo produjo manualmente en 68.5 segundos—terminó en 14.2 segundos por voz, el mayor ahorro relativo con 79.3 %. Retirar a ese participante cambia poco: los seis restantes promedian una reducción del 64.4 %.

5. DISCUSIÓN

5.1 Dos resultados de velocidad que parecen contradecirse

Los dos estudios produjeron hallazgos que parecen opuestos: la voz fue de 2.1 a 2.5 veces *más lenta* que el mouse y el teclado, y 2.80 veces *más rápida* que averiguar el comando. Leídos en conjunto dicen algo más útil que cualquiera de los dos por separado.

Las tres tareas de usabilidad—crear carpetas, mover un archivo, buscar en la web—son tareas que el entorno gráfico ya resuelve bien: un explorador fue construido para eso, los usuarios tienen años de práctica, y dictar seis nombres es genuinamente más lento que escribirlos. La tarea controlada fue de otra naturaleza. Filtrar un directorio por tamaño de archivo es algo que el entorno gráfico no ofrece de forma conveniente, así que la alternativa realista es la terminal, y llegar a ella implica descubrir `find`, luego su sintaxis de tamaño, luego que `-type f` excluye directorios.

Una interfaz conversacional, entonces, no debería plantearse como un reemplazo de la manipulación directa. No compite con el mouse en las tareas para las que el mouse fue diseñado; sustituye a la terminal en las tareas que el entorno gráfico deja sin resolver—precisamente las tareas que hoy excluyen a los usuarios no expertos. Visto así, el 83.9 % de éxito por voz no es insuficiencia frente al 100 % del entorno gráfico, sino una medida de qué tan bien se desempeña una modalidad en el terreno de su rival al primer contacto. Toda medición en ambos estudios se tomó en el primer contacto y sin período de práctica, así que el régimen para el que el nivel de patrones aprendidos está diseñado—la segunda y tercera vez que un usuario pide algo—es uno que estos estudios no alcanzan.

5.2 Accesibilidad, costo y privacidad

La correlación de 0.97 entre tiempo manual y tiempo ahorrado le da al argumento de accesibilidad un sustento empírico del que carecía: quienes más se benefician están más lejos de la línea de comandos, la población que la plataforma hoy excluye. El ahorro absoluto fue mayor para el participante sin experiencia en terminal, y el ahorro relativo mayor para quien ya conocía la respuesta—de modo que el mecanismo no es solo sustitución de conocimiento sino la eliminación del cambio de contexto. El costo y la privacidad interactúan con esto: cincuenta peticiones diarias en la nube cuestan unos dólares al mes, trivial en algunos entornos y un obstáculo en las instituciones que este trabajo tiene como objetivo; el modo sin conexión elimina ese costo y, más importante, la fuga de datos. El precio es precisión y latencia: el modo sin conexión es adecuado para tareas seguras cotidianas y exige la capa de seguridad determinista en lugar de simplemente beneficiarse de ella.

5.3 La elección de modelo como decisión de diseño de interacción

El resultado más transferible del banco de pruebas: dos modelos con precisión al actuar esencialmente igual—87.6 % y 88.9 %—difieren en más de veinte puntos en qué tan seguido actúan, pidiendo `gpt-4o` aclaración tres veces más seguido que `deepseek-chat`. Eso no es una brecha de calidad sino una disposición, y se propaga a la experiencia del usuario: el mismo sistema

se siente autónomo con un modelo y cauteloso con el otro. Para una herramienta construida sobre la premisa de que el usuario mantiene el control, esto es un parámetro de diseño que una sola cifra de precisión oculta por completo, y argumenta a favor de reportar cobertura y precisión por separado siempre que un sistema pueda abstenerse de actuar.

5.4 Limitaciones

1. **El nivel de patrones aprendidos está caracterizado pero no cuantificado.** La Sección 3.2 reporta mecanismo, umbrales y costos, pero no una proporción medida de peticiones resueltas: el arnés corre con el aprendizaje deshabilitado—el valor de fábrica—para aislar la precisión del analizador, así que el 4.4 % de la Sección 4.2 describe solo al primer y tercer nivel. Cuantificarlo requiere uso longitudinal real: es el vacío más significativo de la evaluación.
2. **Tamaños de muestra.** Catorce participantes sustentan hallazgos cualitativos y estadística descriptiva, no inferencia; la comparación entre modelos es exploratoria. La comparación controlada alcanza significancia, pero sus siete pares acotan la precisión de la estimación, no su existencia.
3. **Un solo idioma, e involucramiento del investigador.** Todo esto es en español, con un único modelo acústico de Vosk; el desempeño en otras variantes, y más aún en otros idiomas, no está probado, y el diccionario de anclas es específico del español por construcción. El conjunto de tareas fue compilado por el desarrollador principal, quien también facilitó las sesiones: automatizar el arnés lo retiró de la calificación, no del diseño de lo calificado.
4. **Cobertura y peticiones compuestas.** Se verificaron cuatro de seis familias, y aun ahí solo la instalación. La arquitectura procesa una acción por entrada: la aclaración de un turno maneja parámetros faltantes, pero una petición que encadena operaciones—encontrar archivos grandes, moverlos, comprimirlos—no se descompone en pasos atómicos verificables.
5. **Alcance de la capa de seguridad.** Detecta categorías conocidas de operación peligrosa, no riesgo dependiente del contexto. El modo sin conexión muestra por qué importa: un modelo local pequeño reinterpreto peticiones peligrosas fuera de su repertorio como acciones que sí conocía, y fue la capa determinista, no el modelo, la que se interpuso.

5.5 Trabajo futuro

De aquí se desprenden cuatro direcciones: un despliegue longitudinal que mida qué proporción de peticiones atiende el nivel aprendido en semanas de uso real—la cifra que falta aquí—; la descomposición de peticiones compuestas en un plan previsualizado por completo, autorizado por lote con excepciones por paso para operaciones peligrosas y ejecutado con puntos de verificación, manteniendo cada paso como una acción del repertorio existente; extender la aclaración de un turno a un diálogo; y una comparación cuantitativa contra líneas base como ShellGPT. También pretendemos liberar el sistema como software libre.

6. CONCLUSIONES

Este artículo presentó un recuento extendido de BOTAS, interfaz conversacional que traduce español hablado a comandos válidos de GNU/Linux bajo principios de IA Centrada en el Humano, y reportó la evaluación que la versión de congreso solo podía esbozar. Tres resultados destacan. Primero, en un banco de pruebas de 250 tareas evaluado por un arnés automatizado bajo un criterio estricto de concordancia de acción, el sistema alcanza 78.0 % de cobertura y 87.6 % de precisión cuando actúa, sin falsos negativos de seguridad; la separación entre esas dos cifras importa más que cualquiera por sí sola, dado que un segundo modelo con precisión casi idéntica se abstiene de actuar tres veces más seguido. Segundo, contra la línea base que el sistema realmente desplaza—un usuario no experto averiguando qué escribir—redujo el tiempo de completitud en 66.6 %, un factor de 2.80, con los siete participantes más rápidos y la ventaja preservada bajo contrabalanceo. Tercero, la misma modalidad es de 2.1 a 2.5 veces más lenta que el mouse y el teclado en tareas que el entorno gráfico ya resuelve bien, lo que ubica la contribución con precisión: una interfaz conversacional no es un reemplazo de la manipulación directa sino un sustituto de la terminal en las tareas que la manipulación directa deja sin resolver.

Las extensiones reportadas aquí—un nivel de aprendizaje que convierte cada petición resuelta en plantilla reutilizable, un modo completamente local validado en una GPU de seis gigabytes, y soporte multidistribución verificado mediante instalación desatendida en tres gestores—apuntan en la misma dirección: un sistema que cuesta menos, revela menos y depende menos de la red mientras más se usa. Su limitación central se declara con claridad más arriba: la contribución del nivel de aprendizaje en el mundo real permanece sin medir.

Lo que estos resultados sugieren para la discusión más amplia sobre la IA en la sociedad es más acotado que una afirmación sobre democratizar la computación, y más firme. El beneficio de este tipo de herramienta no es uniforme; escala con la distancia respecto a la experticia, con $r = 0,97$ en nuestros datos. Eso es un argumento para construir este tipo de interfaces donde esa distancia sea mayor, y para medirlas contra la barrera que buscan eliminar y no contra la interfaz que nunca la planteó.

AGRADECIMIENTOS

Los autores agradecen a los participantes de ambos estudios por su tiempo, y a la Universidad Tecnológica de la Mixteca por el apoyo institucional.

REFERENCIAS

- Alpha Cephei. Vosk speech recognition toolkit. Alpha Cephei, 2023.
- S. Amershi et al. Guidelines for human–AI interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, pages 1–13, 2019.
- J. R. Aragon Toledo and R. Ruiz Rodríguez. BOTAS: A human-centered natural language interface for GNU/Linux system management with multi-distribution support. In P. Isaias, P. Miranda, and G. C. Peng, editors, *Proceedings of the International Conferences on Artificial Intelligence in Society 2026, Digital Transformation and Innovation Management 2026 and Connected Smart Cities 2026*, pages 100–110,

- Valencia, Spain, 2026. IADIS Press. ISBN 978-989-8704-78-8.
- F. Bang. GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software*, pages 212–218, 2023.
- A. Berdasco et al. User experience comparison of intelligent personal assistants: Alexa, Google Assistant, Siri and Cortana. *Proceedings of UCAmI 2019*, 31(1):51, 2019.
- V. Braun and V. Clarke. Using thematic analysis in psychology. *Qualitative Research in Psychology*, 3(2): 77–101, 2006.
- T. Brown et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, 2020.
- CEPAL. Panorama digital de América Latina y el Caribe 2023: Avances y desafíos de la transformación digital. Technical report, Comisión Económica para América Latina y el Caribe, Naciones Unidas, 2023.
- L. Dobberstein. Creator of linux virtual assistant blames ‘patent troll’ for project’s death. *The Register*, 2023.
- International Organization for Standardization. ISO 9241-11:2018. ergonomics of human–system interaction — part 11: Usability: Definitions and concepts. Technical report, International Organization for Standardization, Geneva, Switzerland, 2018.
- D. Jakob et al. Adapting voice assistant technology for older adults. *Digital*, 5(1):4, 2025.
- S. Jenson. The future of desktop UX: Breaking the cycle of stagnation. Scott Jenson, personal site, 2024.
- D. Lakens. Calculating and reporting effect sizes to facilitate cumulative science: A practical primer for *t*-tests and ANOVAs. *Frontiers in Psychology*, 4:863, 2013.
- X. V. Lin et al. NL2Bash: A corpus and semantic parser for natural language interface to the Linux operating system. In *Proceedings of the 11th International Conference on Language Resources and Evaluation (LREC 2018)*, 2018.
- E. Luger and A. Sellen. “like having a really bad PA”: The gulf between user expectation and experience of conversational agents. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*, pages 5286–5297, 2016.
- C. D. Manning et al. *Introduction to Information Retrieval*. Cambridge University Press, Cambridge, UK, 2008.
- J. Nielsen. *Usability Engineering*. Morgan Kaufmann, 1994.
- D. A. Norman. *The Design of Everyday Things: Revised and Expanded Edition*. Basic Books, 2013.
- OpenAI. GPT-4 technical report. Technical report, OpenAI, 2023. arXiv:2303.08774.
- OpenVoiceOS. OVOS-core: The FOSS artificial intelligence platform. OpenVoiceOS project repository, 2024.
- A. Radford et al. Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning*, 2023. Preprint 2022.
- S. Saha et al. Jasper: An open source platform for developing always-on, voice-controlled applications. Jasper project repository, 2014.
- F. Shchekotov. ShellGPT: A command-line productivity tool powered by AI large language models. ShellGPT project repository, 2024.
- B. Shneiderman. Human-centered artificial intelligence: Reliable, safe & trustworthy. *International Journal of Human–Computer Interaction*, 36(6):495–504, 2020.
- B. Shneiderman. *Human-Centered AI*. Oxford University Press, 2022.
- B. Shneiderman and C. Plaisant. *Designing the User Interface: Strategies for Effective Human–Computer Interaction*. Addison-Wesley, 5th edition, 2010.
- Top500.org. Operating system family / Linux. TOP500 Project, 2023.

- W3Techs. Usage statistics of Linux for websites. W3Techs Web Technology Surveys, 2023.
- M. Warschauer. *Technology and Social Inclusion: Rethinking the Digital Divide*. MIT Press, 2004.
- J. Wei et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, 2022.
- F. Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.
- J. O. Wobbrock et al. Ability-based design: Concept, principles and examples. *ACM Transactions on Accessible Computing*, 3(3):1–27, 2011.